Assignment 1: Adaline and Logistic Regression

Jackie Javier, Pranitha Achanta, Robert McDaniels
The University of New Mexico
CS429

Due 2/13/2026

1 Task 1

The bias is transformed into an extra weight by adding an additional 1 value to each sample in the training dataset. The weight that will be multiplied by this 1 value is exactly equivalent to the original bias, as bias updates are the same as weight updates (for Adaline and logistic regression), except for the multiplication by the sample. Since the multiplication for the bias will specifically be by a factor of 1, the bias remains unaffected by the sample despite being absorbed by the weight vector.
The line of code that adds a column of ones to the 2D sample array is as follows:

```
1 X_bias = np.hstack([X, np.ones((X.shape[0], 1))])
```

With this update, all references to b_ can be removed. The net_input also remains equivalent, as the bias addition will still occur as $+(b\_ * 1)$.

2 Task 2

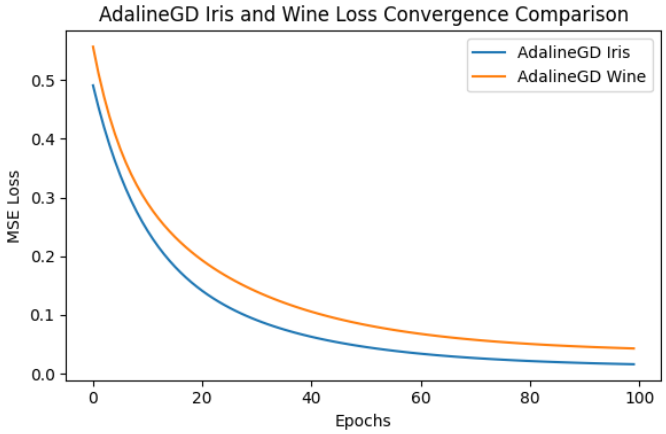

Figure 1

After 100 epochs, Adaline converges on both test sets to 0.05 and 0.025 mean squared error (MSE) loss for the iris and wine datasets, respectively.

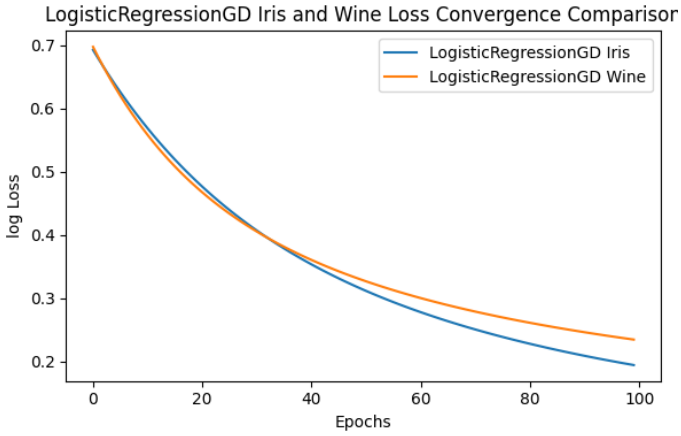

Figure 2

Meanwhile, logistic regression does not reach convergence in the same number of epochs. Further tests indicate that it takes approximately 500 epochs to reach 0.01 and 0.05 log-loss, respectively.

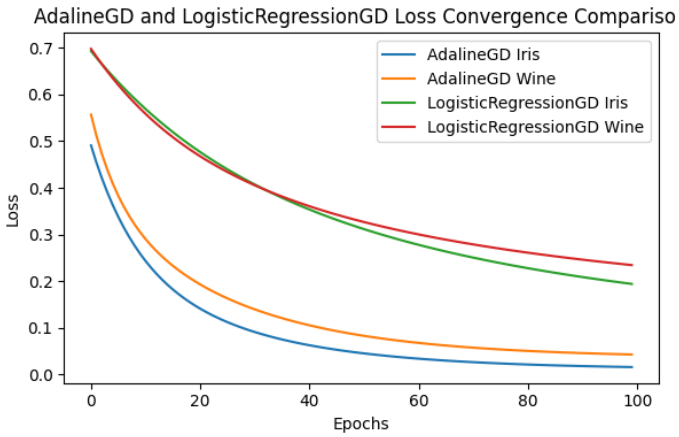

Figure 3

Because Adaline minimizes MSE and logistic regression minimizes log-loss, their loss values are not directly comparable in magnitude and do not directly reflect classification accuracy. However, their convergence behavior across epochs can still be meaningfully compared under identical hyperparameters. From the loss curves, Adaline shows a faster initial decrease and reaches a stable plateau in fewer epochs for both datasets, indicating quicker optimization of its objective function. Logistic regression converges more gradually due to the nonlinear sigmoid activation and the curvature of the log-loss surface.

Thus, Adaline converges faster numerically, while logistic regression optimizes a loss function that is more appropriate for probabilistic binary classification.

3 Task 3

Using a One vs All (OvA) method we fit one model per class, predict independently, and use the result with the highest confidence.

```
setosa = ada_setosa.net_input(X_bias)
versicolor = ada_versicolor.net_input(X_bias)
virginica = ada_virginica.net_input(X_bias)

results = np.column_stack((setosa, versicolor, virginica))
names = np.array(['setosa', 'versicolor', 'virginica'])
predicted_classes = names[np.argmax(results, axis=1)]
```

This composite model can be used to perform multiclass classification using only perceptron binary classifiers. Each binary classifier predicts how likely a sample belongs to its class; the class with the highest output is chosen.

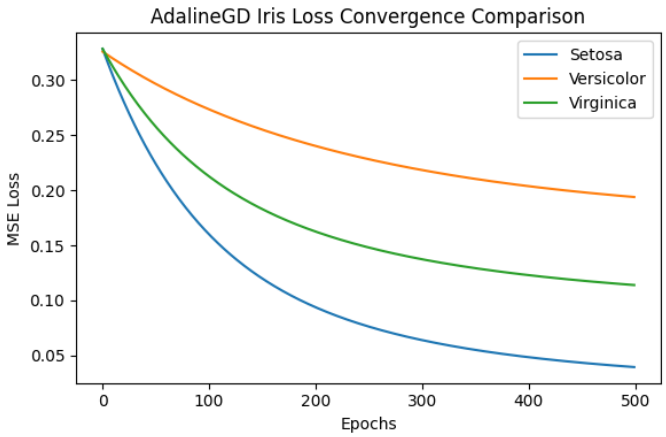

Figure 4: $E = 500, \eta = 0.001$

This loss convergence shows that the classifiers converge at different rates, indicating that the model will be less reliable when predicting those classes. Our results show that versicolor classifications have the most errors, followed by virginica. Setosa has no errors in the test set. This directly coincides with the final losses of each separate model. This result follows known facts about the Iris dataset: setosa is linearly separable from versicolor and virginica, while versicolor and virginica are not linearly separable from each other.

4 Task 4

To implement Stochastic Gradient Descent (SGD), we shuffle the dataset every epoch and apply weight updates iteratively rather than simultaneously. Mini-batch SGD represents a midpoint which iteratively applies GD over smaller mini-batches, converging slower but with better performance.

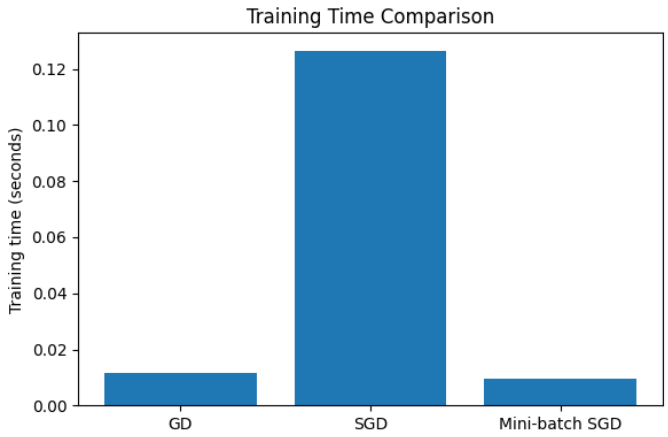

Figure 5

In this implementation, SGD required more training time than GD because weights are updated once per sample, resulting in many more update operations and Python loop overhead. This timing difference reflects the implementation rather than an inherent property of SGD. In optimized machine-learning libraries, SGD is often competitive with or faster than full-batch GD on large datasets. Our implementations of GD and mini-batch are able to take advantage of vectorized matrix multiplication, whereas SGD does not.

Mini-batch SGD required less wall-clock time than GD in this implementation. Because the dataset is small (129 observations), the measured runtimes are heavily influenced by implementation details and library overhead. Consequently, these timing differences should not be interpreted as a general property of the optimization methods. On larger datasets, the relative performance of GD, SGD, and mini-batch SGD depends on the problem size, implementation, and hardware.

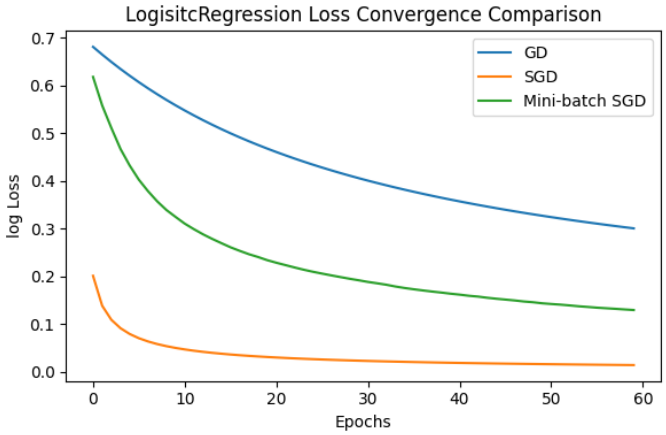

Figure 6: $E = 60$ and $\eta = 0.01$

SGD converges more rapidly than GD and mini-batch SGD because it performs a weight update after every training example, allowing the model to make substantially more optimization steps during each epoch. Mini-batch SGD converges faster than GD but more slowly than SGD, as it updates the weights once per batch rather than once per sample. Although SGD uses noisier gradient estimates based on individual observations, these frequent updates can accelerate progress toward a low-loss solution. In contrast, mini-batch SGD averages gradient information across multiple samples, producing more stable updates and a smoother loss curve. GD exhibits the smoothest convergence because each update is computed from the entire training set, but it performs only a single update per epoch and therefore reduces the loss more gradually. Since loss is evaluated on the full dataset after each epoch for all three methods, the curves are directly comparable representations of model performance over training.

Over 60 epochs, GD performs 60 weight updates, mini-batch SGD performs approximately 300 updates (batch size 32), and SGD performs 7,740 updates. Consequently, SGD achieves the lowest loss after 60 epochs, though this does not necessarily imply greater computational efficiency.